

1 Striktes und Rigoroses 2PL

Welchen Vorteil bietet rigoroses Zwei-Phasen-Sperren? Wie vergleicht es sich mit anderen Formen des Zwei-Phasen-Sperrens?

2 Deadlock Detection

Angenommen, die Transaktionen $T_1 - T_6$ laufen gleichzeitig auf einer Datenbank. Die folgende Tabelle zeigt, welche Datenobjekte $a, \dots, j$ derzeit von den Transaktionen gesperrt sind und auf welche Freigaben von Sperren die Transaktionen warten.

Transaktion	hält Sperre auf	Wartet auf das Freigeben der Sperre auf
T_1	a, b	h
T_2		a, f
T_3	c, d, e	
T_4	f, g	b, c
T_5	h	d, i
T_6	i	e, g

Bitte erstellen Sie den Wait-For Graph und entscheiden Sie, ob ein Deadlock vorliegt. Im Falle eines Deadlocks, lösen Sie ihn bitte auf.

3 Recovery

Drei Transaktionen, T_1, T_2, T_3 , laufen gleichzeitig gemäß der folgenden Tabelle. Die Tabelle zeigt auch die geschriebenen Log-Dateien.

	T_1	T_2	T_3	Log
				[TID, DID, old, new]
1.	BOT			[T_1 start]
2.	$r(A, a_1)$			
3.		BOT		[T_2 start]
4.	$a_1 = a_1 + 4$			
5.		$r(B, b_1)$		
6.		$b_1 = b_1 \cdot 1.1$		
7.		$w(B, b_1)$		[$T_2, B, 1, 1.1$]
				point 1
8.	$w(A, a_1)$			[$T_1, A, 1, 5$]
9.		$r(A, a_2)$		
10.		$a_2 = a_2 + 10$		
11.			BOT	[T_3 start]
12.		$w(A, a_2)$		[$T_2, A, 5, 15$]
13.		commit		[T_2 commit]
14.			$r(B, b_1)$	
15.	$r(C, c_1)$			
16.	$r(B, b_2)$			
17.	$c_1 = c_1 + b_2$			
18.	$w(c_1)$			[$T_1, C, 9, 10.1$]
19.			$b_1 = b_1 \cdot 2$	
20.			$w(B, b_1)$	[$T_3, B, 1.1, 2.2$]
21.	commit			[T_1 commit]
				point 2
22.			commit	[T_3 commit]

1. Das System stürzt an Punkt 1 ab. Geben Sie an, was in jeder Phase des Wiederanlaufs geschieht.
2. Diesmal stürzt das System bei Punkt 1 nicht ab, bei Punkt 2 stürzt es allerdings wieder ab. Geben Sie an, was in jeder Phase des Wiederanlaufs geschieht.

4 Deadlock Prevention vs. Detection

Unter welchen Bedingungen ist es günstiger, (i) Deadlocks zu vermeiden, als (ii) Deadlocks zuzulassen und sie dann zu erkennen?

5 Conflict Graph vs. Wait-For Graph

Beschreiben Sie die Unterschiede und Gemeinsamkeiten zwischen Conflict Graphs und Wait-For Graphs.

Sie können den folgenden Schedules als Grundlage für Ihre Diskussion verwenden:

step	T_1	T_2
1	lock_X(A)	
2	lock_S(B)	
3	write(A)	
4	read(B)	
5	unlock(A)	
6	unlock(B)	
7		lock_X(B)
8		lock_S(A)
9		write(B)
10		read(A)
11		unlock(B)
12		unlock(A)
13	commit	
14		commit